Practical Number		CO/PO
	4. Create a class named AnnotationValidator with a public static method String[] validate(Object obj) that uses reflection — getClass().getDeclaredFields(), field.isAnnotationPresent(...), field.setAccessible(true) and field.get(obj) — to check the @Positive and @MaxLength rules and return the error messages as a String[]. 5. In main, create an Account with a negative balance and run AnnotationValidator.validate() to print the error. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What are the meta-annotations @Retention and @Target used for? 2. What is reflection, and what does getDeclaredFields() return? 3. How do annotations plus reflection let a framework validate any object? **Supplementary Problems –** • Add a @MaxLength(10) rule on a String field and test it. • Make the validator also print the field name with each error message. **Key Skills to be addressed –** Standard annotations (@Override, @FunctionalInterface); meta-annotations; defining custom annotations; reflection on fields and annotations. **Applications –** Validation frameworks, object-relational mappers and dependency-injection containers all read annotations by reflection. **Learning Outcome –** The student can define custom annotations and process them at run time with reflection. (CO1, CO2)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. An Account with an invalid (negative) balance is created in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-07. Confirm that an account with a negative balance produces the @Positive error message and a valid account produces none.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct custom annotations with meta-annotations – 35%; reflection-based validator – 35%; correct use of standard annotations – 10%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: an Account object whose balance is -100. **Process:** validate() reads each field, finds the @Positive annotation on balance, reads the value by reflection, and sees it is not greater than zero. **Expected Output:** The returned String[] contains one message, "must be > 0"; for a valid account the array is empty.	

Practical Number		CO/PO
8	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt. 2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.	**CO3** PO3, PO4, PO5

Practical Number		CO/PO
	3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource. **Part B — Project Work (Hour 2): MiniBank** Until now a failed withdrawal just returned false. In this lab you make MiniBank fault-tolerant: you replace those flags with a custom exception hierarchy so every failure is explicit and carries information (such as the shortfall). This is the Milestone 1 checkpoint, where the model, validation, annotations and error handling come together. 1. In a package named exception, create a checked base class BankException that extends Exception. Then create InsufficientFundsException, AccountNotFoundException and InvalidAmountException, each extending BankException. Give InsufficientFundsException a long field shortfall. 2. Change withdraw(long amount) to declare throws InsufficientFundsException, InvalidAmountException; throw InsufficientFundsException when the balance is too low and InvalidAmountException when the amount is less than or equal to zero. Make deposit(long amount) throw InvalidAmountException for a non-positive amount. 3. Add a method transfer(Account to, long amount) that uses try, catch and finally and re-throws a BankException when needed. 4. In main, wrap the operations in try-catch blocks and print friendly messages; use a finally block; and show one try-with-resources example using a small AutoCloseable class that you create. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between a checked and an unchecked exception? 2. What is the purpose of the finally block, and when does it run? 3. How does try-with-resources guarantee a resource is closed? **Supplementary Problems –** • Add a DailyLimitExceededException and throw it when a withdrawal exceeds a daily cap. • Print the shortfall amount stored inside InsufficientFundsException. **Key Skills to be addressed –** try-catch-finally; checked vs unchecked exceptions; throw and throws; custom exception classes; exception propagation; try-with-resources. **Applications –** Robust error handling is mandatory in banking, payment and any system that must not crash on bad input. **Learning Outcome –** The student can build fault-tolerant code using custom exceptions and structured handling. (CO3)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Operations that trigger each exception are called in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-08. Confirm that withdrawing more than the balance throws InsufficientFundsException and is caught with a clear message. Prepare for the Milestone 1 demo (classes + validation + annotations + exceptions).	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct exception hierarchy – 35%; throw/throws in deposit/withdraw/transfer – 30%; try-catch-finally and try-with-resources – 15%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: an account with balance 1000 on which withdraw(5000) is called.	